

Flex-PGA

Flex-PGA

Sam Kaufman, Madi Licht, Ryan O'Sullivan, Holland Hargens, Prateek Tandon

https://github.com/Ryanos99/ECE554_Capstone

Project Final Report

University of Wisconsin-Madison

Objective

The Flex-PGA project was initiated to address the critical shortcomings of current consumer fitness technology, specifically the dual challenges of computational latency and data privacy. Traditional fitness tracking solutions typically rely on cloud-based processing, which introduces significant "round-trip" delays that disrupt real-time user feedback. Furthermore, streaming live video data of a user's private home environment to external servers raises substantial ethical and security concerns.

Our solution leverages a Xilinx Zynq System-on-Chip (SoC) to create a fully autonomous, "edge-AI" ecosystem. By performing real-time workout recognition and repetition counting locally on the device, we eliminate the need for an external GPU or internet connectivity. The core innovation lies in the transition from general-purpose CPU execution to deterministic hardware acceleration, ensuring that the system provides instantaneous feedback while keeping all sensitive user data confined to the local volatile memory of the FPGA.

Background

2.1 The Limitations of Current Fitness Analytics

The modern fitness technology market is currently dominated by two primary architectural paradigms: smartphone-based local processing and cloud-based inference. While these solutions are accessible, they suffer from inherent structural flaws when applied to real-time bio-mechanical tracking.

Smartphone applications are typically "CPU-heavy," relying on general-purpose ARM processors to handle complex computer vision tasks. Because these processors must manage background OS tasks, thermal throttling, and non-deterministic scheduling, they often struggle to maintain a consistent high frame rate, leading to "dropped counts" and a poor user experience. Conversely, cloud-based servers offer high computational power but introduce significant network latency and jitter. In a high-intensity workout scenario, even a 200ms delay in classification can lead to out-of-sync feedback, rendering the real-time aspect of the tracker ineffective.

2.2 The Case for Edge-AI and Hardware Acceleration

The emergence of "Edge-AI", performing neural network inference directly on the local embedded device, offers a solution to these bottlenecks. However, executing deep learning models at the edge requires massive throughput of Multiply-Accumulate (MAC) operations, which are the fundamental building blocks of both Convolutional Neural Networks (CNNs) and Multi-Layer Perceptrons (MLPs).

Field Programmable Gate Arrays (FPGAs) are uniquely suited for this task compared to standard microcontrollers or CPUs. Unlike the sequential execution model of a CPU, an FPGA can be configured to perform hundreds of MAC operations in parallel by utilizing dedicated Digital Signal Processing (DSP) slices. Furthermore, FPGAs allow for custom bit-precision arithmetic. While standard processors are locked into 32-bit or 64-bit floating-point operations, the Flex-PGA system utilizes 8-bit fixed-point precision. This custom precision drastically reduces power consumption and logic utilization while increasing frames-per-second (FPS) and ensuring the deterministic performance required for accurate, real-time exercise classification.

2.3 Privacy in the IoT Era

Beyond performance, there is a growing ethical demand for privacy-preserving hardware. Standard fitness trackers often require the transmission of sensitive video data or skeletal coordinates to external servers, creating a permanent digital footprint of a user's private activity. By utilizing a Zynq-based SoC architecture, the Flex-PGA project demonstrates a "Local-Only" data model. By keeping the processing pipeline, from camera capture to classification, within the local FPGA fabric and ARM Processing System, we provide a blueprint for a secure, private-by-default fitness ecosystem.

Technical description

3.1 Inference Engine & MLP Hardware Architecture

The final implementation of Flex-PGA utilizes a custom Multi-Layer Perceptron (MLP) alongside a pretrained MoveNet model. This strategic pivot allowed the team to optimize for the Zynq board's specific Logic Cell density while maintaining high classification accuracy for pose-based data.

- **Fixed-Point Optimization:** To maximize hardware throughput, the model was converted from 32-bit floating-point to 8-bit quantized fixed-point integers. This quantization allowed for high-speed matrix multiplication, significantly reducing the computational footprint compared to standard floating-point units.

- **Parallel MAC Array:** The core execution unit consists of a parallel Multiply-Accumulate (MAC) array implemented in Verilog. This array processes input features simultaneously, leveraging the parallel nature of FPGA fabric to achieve deterministic inference speeds that far exceed serial CPU execution.

3.2 PS-PL Co-Design & AXI Infrastructure

The system architecture follows a sophisticated Hardware-Software Co-Design approach, partitioning tasks between the ARM Processing System (PS) and the Programmable Logic (PL) to exploit the strengths of each.

- **The Processing System (ARM Cortex-A9):** The PS functions as the "brain" of the system. It handles high-level I/O tasks, specifically the USB camera interface. The ARM core performs frame capture, resizing, and grayscale normalization before streaming the processed data to the hardware accelerator.
- **The Programmable Logic (FPGA):** The PL serves as the high-speed "muscle," hosting the MLP inference engine and the repetition-counting logic. By offloading these repetitive, mathematically intensive tasks to custom logic, the CPU is freed to manage networking and the user interface.
- **Communication via AXI Interconnect:** Data integrity between the PS and PL is maintained through AXI-Stream and AXI-Lite interfaces. We implemented an AXI-Direct Memory Access controller to facilitate high-speed data transfers, ensuring the video pipeline maintains a consistent 10 FPS without CPU-induced bottlenecks.

3.3 Memory Management & BRAM Integration

Efficient memory access is critical for real-time performance. To avoid the high latency associated with external DDR memory, the Flex-PGA system utilizes on-chip Block RAM (BRAM).

- **Single-Cycle Weight Access:** Model weights are pre-loaded into BRAM during the initialization phase. This allows the inference engine to fetch weights in a single clock cycle, minimizing bus contention and ensuring that the MAC units are never "starved" for data.
- **Dual-Port Buffering:** We utilized dual-port BRAM configurations to allow the PS to update model parameters or thresholds without interrupting the active inference cycles occurring in the PL.

3.4 Hardware Repetition-Counting State Machine

To track workout progress without the overhead of complex Recurrent Neural Networks (RNNs), we engineered a Hardware State Machine (HSM).

- **Deterministic Logic:** The HSM monitors the classification output from the MLP (e.g., *Pushup_Top* or *Pushup_Bottom*). It tracks the specific temporal sequence of these states to identify a completed repetition.
- **Hardware Debouncing:** To mitigate "flicker" caused by classification noise or rapid movement, the state machine implements a confidence counter. It requires a classification to remain stable for a specific number of consecutive frames before triggering a state transition, ensuring high accuracy in rep counting even in suboptimal lighting.

Evaluation highlights

Our system was evaluated based on throughput, accuracy, and resource utilization.

Metric	Target	Achieved
Throughput	15 FPS	10 FPS
Inference Accuracy	>85%	90%
Power Consumption	< 5W	3W
Rep-Counting Error	< 5%	15%

Resource Utilization:

- **BRAM:** 1.62% (Primarily for weight storage)
- **DSP Slices:** 0.28% (Parallel MAC operations)
- **LUTs/FFs:** 3.04% (Control logic and AXI interfaces)

Challenges, workarounds, & lessons learned

Hardware & Documentation Hurdles

- **Challenge:** The team faced significant delays due to broken lab hardware and a total lack of technical documentation for our specific FPGA board. This meant we could not perform the standard "minilabs" to familiarize ourselves with the hardware, leading to weeks of trial-and-error. Furthermore, the MIPI camera interface was very difficult to integrate with the board's physical constraints.
- **Workaround:** We pivoted from the MIPI interface to a USB-based camera interface using UVC drivers. We relied on community forums and reverse-engineering existing sample projects to compensate for the lack of official documentation.

Toolchain & Resource Constraints

- **Challenge:** Institutional constraints prevented the installation of Vivado on lab machines, and the team lacked dedicated Linux workstations required for Petalinux board compilation.
- **Workaround:** We shifted all development to personal laptops, this allowed us to maintain some progress despite the lack of dedicated lab infrastructure.

DPU Integration & Synthesis Bottlenecks

- **Challenge:** We spent several weeks attempting to integrate the Xilinx DPU (Deep Learning Processor Unit), but the board's logic resources (LUTs/BRAM) were too small and different to fit the IP. Additionally, the DPU synthesis cycle took several hours per iteration, making the debug-and-edit loop extremely inefficient.
- **Workaround:** After exhaustive testing, we made the strategic decision to pivot away from the DPU towards running it on the PS.

Memory Optimization & Model Accuracy

- **Challenge:** Limited on-chip memory meant we could not store high-precision weights for a complex model, creating a bottleneck that threatened our 15 FPS goal.
- **Workaround:** We implemented aggressive weight quantization and model pruning. While this initially led to a drop in accuracy, it allowed the model to fit within the BRAM constraints. We compensated for the accuracy loss by fine-tuning the classification thresholds in the software post-processing layer.

Future Work & Scalability

While the current implementation successfully demonstrates an MLP-based workout classifier, the architecture is designed for future expansion. With a higher-density FPGA board the system could be upgraded to run full Pose Estimation models like MoveNet or OpenPose entirely within the Programmable Logic. Additionally, we could expand the number of workouts we classify, add user tracking, or improve classification accuracy. There are many potential options of how this project could be continued and approved in the future.